

```

import cv2
import numpy as np
import os
from IPython.display import Video, display

VIDEO_PATH = "/content/stock-footage--k-nature-loop-able-sec-video-waterfall-over-rock"
RESIZE_WIDTH = 480 # lower = faster processing

if not os.path.exists(VIDEO_PATH):
    raise FileNotFoundError(f"Video not found at {VIDEO_PATH}. Upload it to Colab first")

def resize_frame(frame):
    h, w = frame.shape[:2]
    scale = RESIZE_WIDTH / w
    return cv2.resize(frame, (RESIZE_WIDTH, int(h * scale)))

def run_lucas_kanade(source, output_path="lucas_kanade_output.mp4"):
    cap = cv2.VideoCapture(source)
    fps = cap.get(cv2.CAP_PROP_FPS) or 20
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    ret, old_frame = cap.read()
    if not ret:
        print("Error: Cannot read video source.")
        return
    old_frame = resize_frame(old_frame)
    h, w = old_frame.shape[:2]
    out = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))

    feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7)
    lk_params = dict(winSize=(15, 15), maxLevel=2,
                      criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.01))
    color = np.random.randint(0, 255, (100, 3))

    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
    p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
    mask = np.zeros_like(old_frame)

    frame_num = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame = resize_frame(frame)
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)

        if p1 is not None:
            good_new = p1[st == 1]
            good_old = p0[st == 1]
            for i, (new, old) in enumerate(zip(good_new, good_old)):
                a, b = new.ravel()
                c, d = old.ravel()

```

```

        mask = cv2.line(mask, (int(a), int(b)), (int(c), int(d)), color[i].tolist(), -1)
        frame = cv2.circle(frame, (int(a), int(b)), 5, color[i].tolist(), -1)

    img = cv2.add(frame, mask)
    out.write(img)

    old_gray = frame_gray.copy()
    if p1 is not None:
        p0 = good_new.reshape(-1, 1, 2)
    if p0 is None or len(p0) < 10:
        p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
        mask = np.zeros_like(old_frame)

    frame_num += 1
    if frame_num % 20 == 0:
        print(f"Lucas-Kanade: {frame_num}/{total} frames")

cap.release()
out.release()
print(f"Saved: {output_path}")

def run_farneback(source, output_path="farneback_output.mp4"):
    cap = cv2.VideoCapture(source)
    fps = cap.get(cv2.CAP_PROP_FPS) or 20
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    ret, frame1 = cap.read()
    if not ret:
        print("Error: Cannot read video source.")
        return
    frame1 = resize_frame(frame1)
    h, w = frame1.shape[:2]
    out = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))

    prev_gray = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

    frame_num = 0
    while True:
        ret, frame2 = cap.read()
        if not ret:
            break
        frame2 = resize_frame(frame2)
        next_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

        flow = cv2.calcOpticalFlowFarneback(
            prev_gray, next_gray, None,
            pyr_scale=0.5, levels=3, winsize=15,
            iterations=3, poly_n=5, poly_sigma=1.2, flags=0
        )

        magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])
        hsv[..., 0] = angle * 180 / np.pi / 2
        hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
        bgr_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

```

```

        out.write(bgr_flow)
        prev_gray = next_gray

    frame_num += 1
    if frame_num % 20 == 0:
        print(f"Farneback: {frame_num}/{total} frames")

    cap.release()
    out.release()
    print(f"Saved: {output_path}")

run_lucas_kanade(VIDEO_PATH)
run_farneback(VIDEO_PATH)

print("Lucas-Kanade Output:")
display(Video("lucas_kanade_output.mp4", embed=True))

print("Farneback Output:")
display(Video("farneback_output.mp4", embed=True))

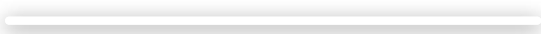
```

```

Lucas-Kanade: 20/126 frames
Lucas-Kanade: 40/126 frames
Lucas-Kanade: 60/126 frames
Lucas-Kanade: 80/126 frames
Lucas-Kanade: 100/126 frames
Lucas-Kanade: 120/126 frames
Saved: lucas_kanade_output.mp4
Farneback: 20/126 frames
Farneback: 40/126 frames
Farneback: 60/126 frames
Farneback: 80/126 frames
Farneback: 100/126 frames
Farneback: 120/126 frames
Saved: farneback_output.mp4
Lucas-Kanade Output:

```

0:00



Farneback Output:

0:00



## Theory Questions & Answers

1. What is Optical Flow? How is it used in computer vision? Optical flow is the pattern of apparent motion of pixel intensities between two consecutive frames, caused by the relative movement of objects and the camera. It's used for motion detection, object tracking, video compression, autonomous navigation, and action/gesture recognition.
2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm. Lucas-Kanade is a sparse, local method. It first selects a set of good feature points (e.g., corners via Shi-Tomasi). For each point, it assumes the flow is essentially constant within a small neighborhood (window) around it, and that brightness is conserved between frames. This gives an over-determined system of the optical flow equation, which is solved using least squares to estimate the  $(u, v)$  displacement of each tracked point across frames using image gradients  $(I_x, I_y, I_t)$ .
3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow? Dense optical flow (e.g., Farneback) computes a motion vector for every pixel in the frame, giving a complete motion field. Sparse optical flow (e.g., Lucas-Kanade) computes motion only for a selected set of distinctive feature points. Dense flow is more computationally expensive but gives full-scene motion information; sparse flow is faster and lighter but only tracks specific points.
4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

| Aspect                                     | Lucas-Kanade | Farneback | Type | Sparse | Dense | Computation | Fast | tracks few points |
|--------------------------------------------|--------------|-----------|------|--------|-------|-------------|------|-------------------|
| Slower                                     |              |           |      |        |       |             |      |                   |
| tracks all pixels                          |              |           |      |        |       |             |      |                   |
| Output                                     |              |           |      |        |       |             |      |                   |
| Motion of specific features                |              |           |      |        |       |             |      |                   |
| Full motion field                          |              |           |      |        |       |             |      |                   |
| Use case                                   |              |           |      |        |       |             |      |                   |
| Object/feature tracking                    |              |           |      |        |       |             |      |                   |
| Scene-wide motion analysis                 |              |           |      |        |       |             |      |                   |
| Robustness to large motion                 |              |           |      |        |       |             |      |                   |
| Handled via pyramids (limited)             |              |           |      |        |       |             |      |                   |
| Handled via pyramids, polynomial expansion |              |           |      |        |       |             |      |                   |

5. What assumptions are made while computing optical flow?

Brightness constancy: pixel intensity of a point doesn't change between frames. Temporal persistence (small motion): motion between consecutive frames is small. Spatial coherence: neighboring pixels belong to the same surface and move similarly.

6. What factors can affect the accuracy of optical flow estimation? Fast object/camera motion, illumination changes, occlusion, lack of texture (uniform regions — the aperture problem), noise, motion blur, and non-rigid deformations.
7. Mention five real-world applications of optical flow in computer vision. Autonomous driving (obstacle/motion detection), video surveillance, robotics navigation, gesture/action recognition, sports analytics (player/ball tracking), video compression, and augmented reality.
8. Why are grayscale images generally used for optical flow computation? Optical flow relies on intensity gradients rather than color, and grayscale reduces computation (single channel vs. three), speeding up gradient calculations without losing the motion information needed.

9. What are the limitations of optical flow algorithms in real-world environments? They struggle with large/fast displacements, illumination changes (violating brightness constancy), occlusions, textureless regions (aperture problem), noise, and non-rigid or transparent object motion.
10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition? In autonomous driving, it helps estimate ego-motion and detect moving obstacles/pedestrians for collision avoidance. In video surveillance, it detects unusual motion or intrusions in a monitored scene. In action recognition, motion vectors capture temporal dynamics of body/limb movement, which are used as features to classify human actions

...

Ellipsis

Start coding or [generate](#) with AI.